Connect++ MVP — Environment Setup & Sprint 1 Foundation

Context

You are building "Connect++", an AI-native service delivery platform for FullStack (a software engineering services company). Connect++ replaces the traditional outsourcing procurement cycle with a software experience where SMB customers can procure engineering services in four modes: individual Talent, Pods (cross-functional teams), Predefined Outcomes (productized services at fixed prices), and Custom Outcomes.

This needs to be demo-ready by Wednesday April 15, 2026. We are building the real product, not a throwaway prototype — but speed matters. Make pragmatic decisions.

Part 1: Environment Setup

Tech Stack

- **Framework:** Next.js 14+ (App Router)
- **Language:** TypeScript (strict mode)
- **Styling:** Tailwind CSS 3.4+
- **UI Components:** shadcn/ui (install via CLI — we will customize heavily later)
- **Database & Auth:** Supabase (use the JS client library `@supabase/supabase-js` v2)
- **Deployment:** Vercel (configure for preview deploys on PR, production on main)
- **Package manager:** pnpm

Project Scaffold

1. Initialize a new Next.js project with App Router, TypeScript, Tailwind, and ESLint.
2. Install and initialize shadcn/ui. Add these components upfront: `button`, `card`, `input`, `textarea`, `badge`, `dialog`, `tabs`, `separator`, `avatar`, `dropdown-menu`, `select`, `progress`, `sheet`, `toast`.
3. Install additional dependencies: `@supabase/supabase-js`, `@supabase/ssr`, `lucide-react`, `date-fns`, `zod` (for form validation), `zustand` (lightweight client state).
4. Set up the Supabase client:
    - Create `lib/supabase/client.ts` (browser client)
    - Create `lib/supabase/server.ts` (server component client using cookies)
    - Create `lib/supabase/middleware.ts` (auth middleware for protected routes)

- Use environment variables: `NEXT_PUBLIC_SUPABASE_URL` and `NEXT_PUBLIC_SUPABASE_ANON_KEY`

5. Create a `.env.local.example` file with all required env vars documented.
6. Set up Next.js middleware (`middleware.ts` at project root) that:
   - Refreshes the Supabase auth session on every request
   - Redirects unauthenticated users away from `/dashboard/*` routes to `/login`
   - Redirects authenticated users away from `/login` and `/signup` to `/dashboard`

Folder Structure

```
src/
├── app/
|   ├── (auth)/
|   |   ├── login/page.tsx
|   |   └── signup/page.tsx
|   ├── (marketing)/
|   |   ├── page.tsx                  # Landing / home page
|   |   └── marketplace/
|   |       ├── page.tsx              # Marketplace browse (all 4 modes)
|   |       ├── outcomes/
|   |       |   ├── page.tsx          # Predefined Outcomes catalog
|   |       |   └── [slug]/
|   |       |       └── page.tsx      # Individual outcome detail + intake
|   |       └── talent/
|   |           └── page.tsx          # Talent browse
|   ├── dashboard/
|   |   ├── layout.tsx                # Authenticated dashboard shell
|   |   ├── page.tsx                  # Engagement overview (all active)
|   |   └── engagements/
|   |       └── [id]/
|   |           ├── page.tsx          # Engagement detail (milestones)
|   |           └── messages/page.tsx # Communication hub for engagement
|   ├── layout.tsx                    # Root layout
|   └── globals.css
├── components/
|   ├── ui/                           # shadcn components (auto-generated)
|   ├── layout/
|   |   ├── marketing-header.tsx
|   |   ├── marketing-footer.tsx
|   |   ├── dashboard-sidebar.tsx
|   |   └── dashboard-header.tsx
```

```
│   ├── marketplace/
│   │   ├── outcome-card.tsx
│   │   ├── outcome-catalog.tsx
│   │   ├── talent-card.tsx
│   │   ├── intake-form.tsx           # Dynamic intake questionnaire
│   │   └── scope-estimate.tsx        # Generated scope display
│   └── dashboard/
│       ├── engagement-card.tsx
│       ├── engagement-list.tsx
│       ├── milestone-tracker.tsx
│       ├── message-thread.tsx
│       └── status-badge.tsx
├── lib/
│   ├── supabase/
│   │   ├── client.ts
│   │   ├── server.ts
│   │   └── middleware.ts
│   ├── types.ts                      # Shared TypeScript types
│   ├── constants.ts                  # Outcome templates, pod configs, etc.
│   └── utils.ts                      # Helpers
└── hooks/
    └── use-supabase.ts               # Custom hook for client-side queries
```

Part 2: Database Schema

Create a Supabase SQL migration file (`supabase/migrations/001_initial_schema.sql`) with the following tables. Use UUIDs for all primary keys. Enable Row Level Security (RLS) on every table.

Tables

```sql
```

```sql
-- Companies (the customer's organization)
create table companies (
  id uuid primary key default gen_random_uuid(),
  name text not null,
  website text,
  size text check (size in ('1-10', '11-50', '51-200', '201-500', '500+')),
  industry text,
  created_at timestamptz default now(),
  updated_at timestamptz default now()
);

-- Users (customer-side users who log into Connect++)
create table users (
  id uuid primary key references auth.users(id) on delete cascade,
  company_id uuid references companies(id),
  full_name text not null,
  email text not null unique,
  role text default 'member' check (role in ('owner', 'admin', 'member')),
  avatar_url text,
  created_at timestamptz default now()
);

-- Outcome Templates (the productized service catalog)
create table outcome_templates (
  id uuid primary key default gen_random_uuid(),
  slug text unique not null,
  title text not null,
  subtitle text,
  description text not null,
  category text not null check (category in ('build', 'automate', 'migrate', 'optimi
  price_range_low integer,         -- in cents
  price_range_high integer,        -- in cents
  timeline_range_low integer,      -- in business days
  timeline_range_high integer,     -- in business days
  icon text,                       -- lucide icon name
  features jsonb default '[]',     -- array of feature strings
  intake_schema jsonb not null,    -- JSON schema defining the intake questionnaire
  is_active boolean default true,
  display_order integer default 0,
  created_at timestamptz default now()
);

-- Engagements (an active or completed piece of work)
```

```
create table engagements (
  id uuid primary key default gen_random_uuid(),
  company_id uuid references companies(id) not null,
  template_id uuid references outcome_templates(id),  -- null for talent/custom
  mode text not null check (mode in ('talent', 'pod', 'predefined_outcome', 'custom_
  title text not null,
  status text default 'intake' check (status in ('intake', 'scoping', 'active', 'in_
  intake_responses jsonb,          -- customer's answers to intake questionnaire
  scope_summary text,              -- PM-validated scope description
  price_cents integer,
  start_date date,
  target_end_date date,
  actual_end_date date,
  created_at timestamptz default now(),
  updated_at timestamptz default now()
);

-- Milestones (delivery checkpoints within an engagement)
create table milestones (
  id uuid primary key default gen_random_uuid(),
  engagement_id uuid references engagements(id) on delete cascade not null,
  title text not null,
  description text,
  status text default 'upcoming' check (status in ('upcoming', 'in_progress', 'in_re
  deliverables jsonb default '[]', -- array of { name, description, status }
  due_date date,
  completed_at timestamptz,
  display_order integer default 0,
  created_at timestamptz default now()
);

-- Messages (communication hub)
create table messages (
  id uuid primary key default gen_random_uuid(),
  engagement_id uuid references engagements(id) on delete cascade not null,
  user_id uuid references users(id),
  sender_name text not null,       -- denormalized for display
  sender_role text default 'client' check (sender_role in ('client', 'pm', 'engineer
  content text not null,
  is_system_message boolean default false,
  created_at timestamptz default now()
);

-- Talent Profiles (engineers available for hire)
```

```sql
create table talent_profiles (
  id uuid primary key default gen_random_uuid(),
  display_name text not null,
  title text not null,             -- e.g. "Senior Full-Stack Engineer"
  seniority text check (seniority in ('mid', 'senior', 'staff', 'principal')),
  bio text,
  skills jsonb default '[]',      -- array of skill strings
  ai_velocity_score numeric(3,1), -- e.g. 2.3 (meaning 2.3x)
  years_experience integer,
  avatar_url text,
  hourly_rate_cents integer,
  is_available boolean default true,
  highlight_projects jsonb default '[]', -- array of { title, description, tech }
  created_at timestamptz default now()
);
```

RLS Policies

Write basic RLS policies:

- `companies`: users can read/update their own company
- `users`: users can read their own record and other users in their company
- `outcome_templates`: anyone can read active templates (public catalog)
- `engagements`: users can read engagements belonging to their company
- `milestones`: users can read milestones for their company's engagements
- `messages`: users can read and insert messages for their company's engagements
- `talent_profiles`: anyone can read available talent (public catalog)

Part 3: Seed Data

Create a seed file (`supabase/seed.sql` or a TypeScript seed script) with the following data:

5 Outcome Templates

1. MVP Sprint

- Category: build
- Slug: mvp-sprint
- Subtitle: "From idea to working product in 3 weeks"

- Description: "Get a production-ready MVP built by an AI-native engineering team. You bring the product brief — we handle architecture, design, development, testing, and deployment. Includes one round of revisions and 30-day post-launch support."
- Price range: $15,000 – $35,000
- Timeline: 15–21 business days
- Features: ["Product architecture & tech stack selection", "UI/UX design (up to 10 screens)", "Full-stack development", "Automated test suite", "CI/CD pipeline & production deployment", "30-day post-launch warranty"]
- Intake schema (questionnaire fields):
  - what_building (textarea): "Describe what you're building — the more detail the better. Upload a PRD, wireframes, or even a napkin sketch if you have one."
  - platform (select: "Web", "iOS", "Android", "Cross-platform"): "What platform?"
  - user_type (select: "Internal team", "B2B customers", "B2C consumers", "Marketplace"): "Who are the primary users?"
  - scale (select: "<100", "100–1K", "1K–10K", "10K+"): "Expected users at launch?"
  - has_designs (select: "Yes, Figma/Sketch files", "Rough wireframes", "Nothing — include design"): "Do you have existing designs?"
  - integrations (multiselect: "Payment processing", "Auth/SSO", "Third-party APIs", "CRM", "None yet"): "Key integrations needed?"
  - timeline (select: "ASAP", "4–6 weeks", "Flexible"): "Timeline preference?"

2. Automated Testing Setup

- Category: automate
- Slug: automated-testing
- Subtitle: "Ship with confidence — full test coverage in 1–2 weeks"
- Description: "We analyze your codebase and build a comprehensive automated test suite — unit, integration, and end-to-end — integrated into your CI pipeline. Includes coverage targets and a testing strategy document your team can maintain."
- Price range: $8,000 – $20,000
- Timeline: 5–10 business days
- Features: ["Codebase audit & test strategy", "Unit test suite", "Integration tests for critical paths", "E2E tests (Playwright or Cypress)", "CI pipeline integration", "Coverage reporting dashboard", "Testing best practices documentation"]
- Intake: repo_url, framework (React/Vue/Angular/Node/Python/Other), current_coverage (None/Under 20%/20-50%/50%+), ci_tool (GitHub Actions/GitLab

CI/CircleCI/Jenkins/None), priority_areas (textarea)

3. Landing Page & Marketing Site

- Category: build
- Slug: landing-page
- Subtitle: "Designed, built, and deployed in 5–10 days"
- Description: "A high-converting marketing site built on modern web tech with CMS integration. Responsive, fast, SEO-optimized, and ready to capture leads from day one."
- Price range: $5,000 – $15,000
- Timeline: 5–10 business days
- Features: ["Custom responsive design", "CMS integration (manage content yourself)", "SEO optimization & meta tags", "Analytics setup (GA4 / Plausible)", "Contact form & lead capture", "Performance optimized (95+ Lighthouse score)"]
- Intake: brand_guidelines (yes/no + upload), pages_needed (select: "Single page", "3–5 pages", "5–10 pages"), cms_preference (select: "Contentful", "Sanity", "WordPress", "No preference"), reference_sites (textarea), goals (textarea: "What's the primary goal of the site?")

4. CI/CD Pipeline Build

- Category: automate
- Slug: cicd-pipeline
- Subtitle: "Zero to fully automated deploys in 1–2 weeks"
- Description: "We set up your entire build-test-deploy pipeline from scratch. Automated testing on every PR, staging environments for review, and one-click production deploys. Includes infrastructure-as-code so your team can maintain and extend it."
- Price range: $6,000 – $18,000
- Timeline: 5–10 business days
- Features: ["Automated build & test on every PR", "Staging environment with preview deploys", "Production deploy pipeline", "Infrastructure-as-code (Terraform / Pulumi)", "Secrets management", "Monitoring & alerting setup", "Runbook documentation"]
- Intake: cloud_provider (AWS/GCP/Azure/Other), repo_host (GitHub/GitLab/Bitbucket), current_deploy_process (textarea), environments_needed (multiselect: "Dev", "Staging", "Production", "QA"), app_type (select: "Web app", "API/Backend", "Mobile backend", "Monorepo")

5. Performance Audit & Fix

- Category: optimize
- Slug: performance-audit
- Subtitle: "Find the bottlenecks. Fix them. Measure the improvement."
- Description: "Our engineers profile your application under realistic load, identify the top performance bottlenecks, and fix them — with measurable before/after benchmarks. You get a faster app and a performance playbook."
- Price range: $8,000 – $22,000
- Timeline: 5–15 business days
- Features: ["Load testing & profiling under realistic traffic", "Bottleneck identification & root cause analysis", "Top 5–10 fixes implemented", "Before/after benchmark report", "Performance monitoring setup", "Optimization playbook for ongoing maintenance"]
- Intake: app_url (text), tech_stack (textarea), current_issues (textarea: "What performance problems are you seeing?"), traffic_volume (select: "<1K req/min", "1K–10K", "10K–100K", "100K+"), target_metrics (textarea: "What does 'fast enough' look like?")

8 Talent Profiles (fictional)

Create 8 diverse engineer profiles with varied skills (React, Node, Python, Go, iOS, Android, DevOps, ML), seniority levels, AI Velocity Scores ranging from 1.8x to 2.7x, and 1–2 highlight projects each. Give them realistic first names only (no last names — privacy). Make them feel like real senior engineers, not marketing copy.

Demo Engagement Data

Create 2 sample engagements with milestones for the demo:

Engagement 1: "Acme Corp — Inventory Management MVP"

- Mode: predefined_outcome, template: MVP Sprint
- Status: active
- 4 milestones: Architecture & Design (completed), Core Build (in_progress), Polish & QA (upcoming), Launch & Handoff (upcoming)
- 5–8 messages showing a realistic PM ↔ client conversation thread

Engagement 2: "Bloom Health — Automated Testing Setup"

- Mode: predefined_outcome, template: Automated Testing
- Status: in_review (milestone 2 awaiting client approval)

- 3 milestones: Codebase Audit (completed), Test Suite Build (in_review), CI Integration (upcoming)
- 3–5 messages

Part 4: Sprint 1 Implementation

Build these pages and components. Every page should be functional with real data from Supabase (use the seed data). Use server components by default; add `"use client"` only where interactivity requires it.

4.1 — Marketing Layout & Home Page (`/(marketing)/page.tsx`)

A clean, modern landing page for Connect++. This is the first thing prospects see.

Sections:

1. **Hero:** Headline "Engineering outcomes, not headcount." Subline: "Connect++ is the AI-native platform where you procure engineering services — from individual talent to complete product builds — with full transparency and measurable velocity." Two CTAs: "Browse Outcomes" (→ /marketplace/outcomes) and "Explore Talent" (→ /marketplace/talent).
2. **How It Works:** 4-step visual: Choose Your Mode → Configure Scope → We Build (AI-Accelerated) → Track Everything in Real Time.
3. **The Four Modes:** Cards for Talent, Pods, Predefined Outcomes, Custom Outcomes. Each with icon, short description, and CTA. Pods and Custom Outcomes show "Coming Soon" badges.
4. **Social proof strip:** "Trusted by 600+ companies including..." with placeholder logos.
5. **CTA footer:** "Ready to ship faster?" with sign-up button.

Design notes: Match the fullstack.com brand identity exactly. Dark background (`#0B0B0F`), bright neon green accent (`#A6F84C`) for primary CTAs and highlights, clean white text. The FullStack site uses bold, confident typography with generous spacing. Hero should feel expansive — large type, plenty of breathing room. CTAs are solid neon green with dark text. Cards have subtle dark borders (`#1E1E24`) with slight hover lifts. Reference fullstack.com/talent/home for the card and section rhythm. Include the FullStack logo (white horizontal SVG from cdn.prod.website-files.com/63ea859d3ade03089d7e65c6/651c3035ed3443430da378d1_fs_logo_horizontal_white.svg) in the header — but append "Connect++" as a wordmark next to it to establish the sub-brand.

4.2 — Predefined Outcomes Catalog (`/(marketing)/marketplace/outcomes/page.tsx`)

Layout:

- Category filter tabs across the top: All, Build, Automate, Migrate, Optimize
- Grid of outcome cards (use `outcome-card.tsx` component)
- Each card shows: icon, title, subtitle, price range, timeline range, category badge, and a "Learn More →" link

Data: Fetch from `outcome_templates` table (server component, no loading state needed for SSR).

4.3 — Outcome Detail & Intake (`/(marketing)/marketplace/outcomes/[slug]/page.tsx`)

Layout:

- Left column (60%): Template title, description, features list, "What's Included" section, timeline visualization
- Right column (40%): Sticky pricing card showing price range, timeline range, and "Start This Project →" button
- Below: Intake form (rendered from the template's `intake_schema` JSON). Each field type (text, textarea, select, multiselect) renders the appropriate shadcn component.
- Submit button: "Submit for Scoping" — on submit, create an `engagements` record in Supabase with status `intake` and the form responses in `intake_responses`. Show a confirmation screen: "We've received your project details. An AI-native Project Manager will review your scope and reach out within 24 hours."
- Note: For the MVP, intake submission should work even for unauthenticated users (capture email in the form). If the user is authenticated, associate with their company.

4.4 — Talent Browse (`/(marketing)/marketplace/talent/page.tsx`)

Layout:

- Filter bar: skill tags (clickable to filter), seniority dropdown, availability toggle
- Grid of talent cards (use `talent-card.tsx`)
- Each card: avatar placeholder (initials-based), name, title, seniority badge, top 3–4 skill tags, AI Velocity Score (prominently displayed, e.g., "2.3x" with a small lightning icon), years of experience, hourly rate range
- Click a card → expand inline or open a sheet with full bio, highlight projects, and "Request This Engineer" CTA (links to sign up / contact flow for MVP)

4.5 — Auth Pages (`/(auth)/login` and `/(auth)/signup`)

Use Supabase Auth with email + password for MVP. Clean, minimal forms.

Sign Up flow:

1. Email, password, full name, company name
2. On submit: create auth user → create company record → create user record → redirect to /dashboard
3. Handle errors gracefully (email taken, weak password, etc.)

Login: Email + password → redirect to /dashboard.

Design: Same dark aesthetic as marketing pages. Centered card layout. Neon green submit button. FullStack logo above the form.

4.6 — Dashboard Layout (`/dashboard/layout.tsx`)

Sidebar navigation:

- Logo (Connect++ wordmark)
- Nav items: Overview (home icon), Engagements (layers icon), Marketplace (store icon), Messages (message-circle icon), Settings (gear icon — placeholder, no page needed)
- Bottom: user avatar + name + company, sign out button

Top bar:

- Page title (dynamic based on route)
- "New Engagement" button (links to /marketplace/outcomes)
- Notification bell (placeholder, non-functional)

Design: Dark sidebar (`#0B0B0F`) with neon green active indicator on selected nav item. Slightly lighter main content area (`#111116`). Compact but not cramped. Matches the fullstack.com dark aesthetic throughout.

4.7 — Dashboard Overview (`/dashboard/page.tsx`)

Sections:

1. **Welcome banner:** "Welcome back, {name}. Here's what's happening across your engagements."
2. **Stats row:** 3–4 cards showing aggregate metrics: Active Engagements count, Milestones Completed This Month, Messages (unread count placeholder), Overall Health

(green/yellow/red summary)

3. **Active Engagements list:** Cards for each active engagement (use `engagement-card.tsx`). Each shows: title, mode badge, status badge, progress bar (milestones completed / total), next milestone name + due date, last message preview. Click → navigate to `/dashboard/engagements/[id]`.
4. **Empty state:** If no engagements, show a friendly prompt: "No active engagements yet. Browse our outcome catalog to get started." with CTA.

4.8 — Engagement Detail (`/dashboard/engagements/[id]/page.tsx`)

Tabs: Overview | Milestones | Messages

Overview tab:

- Engagement title, mode, status, template name
- Key dates: started, target end, current phase
- Scope summary (from `scope_summary` field)
- Price (if applicable)
- Quick stats: milestones completed, messages exchanged, days elapsed

Milestones tab (use `milestone-tracker.tsx`):

- Vertical timeline layout (not Gantt — simpler for MVP)
- Each milestone is a card on the timeline: title, description, status badge, due date, deliverables checklist
- Visual connectors between milestones (line or dots)
- Current milestone highlighted / expanded by default
- Status colors: upcoming (gray `#6B7280`), in_progress (brand green `#A6F84C`), in_review (amber `#FBBF24`), completed (emerald `#34D399`)

Messages tab (use `message-thread.tsx`):

- Chronological message thread (like Slack or iMessage — not email)
- Each message shows: sender avatar (initials), sender name, role badge (Client / PM / System), timestamp, content
- System messages styled differently (centered, muted, smaller — e.g., "Milestone 1 marked as completed")
- Message input at bottom: textarea + send button. On submit, insert into `messages` table. Use optimistic UI update.

- For MVP, messages are not real-time (refresh to see new messages). Real-time via Supabase subscriptions is a fast follow.

Part 5: Design System — FullStack Brand Alignment

Connect++ is a sub-brand of FullStack. It must look and feel like it belongs on fullstack.com — same dark palette, same neon green accent, same confident typography. A user moving between fullstack.com and Connect++ should feel zero brand discontinuity.

Brand Reference

- **Primary reference:** https://www.fullstack.com (home page hero, card styles, footer)
- **Secondary reference:** https://www.fullstack.com/talent/home (talent cards, step-by-step layout, comparison table)
- **Logo:** Use the white horizontal FullStack logo (from the site's CDN) with "Connect++" appended as a wordmark in the same weight. In the dashboard sidebar, use a compact version.

Color Palette (CSS custom properties in globals.css)

- Background primary: `#0B0B0F` (the near-black used across fullstack.com)
- Background secondary: `#111116` (slightly lighter — dashboard content area, card interiors)
- Surface / cards: `#16161C` (card backgrounds, sidebar)
- Surface elevated: `#1E1E24` (hover states, active items, raised cards)
- Border: `#2A2A30` (subtle borders on cards and dividers — not pure gray, slightly warm)
- Text primary: `#FFFFFF` (pure white for headlines — fullstack.com uses bold white headings)
- Text secondary: `#9CA3AF` (muted body text and labels — gray-400 range)
- Text tertiary: `#6B7280` (disabled, timestamps, subtle metadata)
- **Accent / Brand Green:** `#A6F84C` (the signature neon green used for CTAs on fullstack.com — primary buttons, active states, links, highlights, progress bars)
- Accent hover: `#BCFF6E` (lighter green on hover)
- Accent pressed: `#8FD940` (slightly darker on press)
- Accent text: `#0B0B0F` (dark text on green buttons — high contrast)
- Success: `#34D399` (emerald — distinct from brand green, used for status only)
- Warning: `#FBBF24` (amber)

- Danger: `#F87171` (red-400)
- Info / In Progress: `#60A5FA` (blue-400)
- Mode badges: Talent = `#A78BFA` (violet), Pods = `#F472B6` (pink), Outcomes = `#A6F84C` (brand green), Custom = `#FB923C` (orange)

Typography

- **Headings:** `"Outfit", sans-serif` (import from Google Fonts) — this is a geometric sans-serif that closely matches the confident, clean heading style on fullstack.com. Use weights 600–700 for headings.
- **Body:** `"Inter", sans-serif` (import from Google Fonts) — highly legible at small sizes, works well for dashboard content, form labels, descriptions. Use weights 400–500.
- **Metrics / Data / Code:** `"JetBrains Mono", monospace` (import from Google Fonts) — for AI Velocity Scores, prices, timelines, code references, and any numerical data. This adds a technical edge that reinforces the engineering brand.
- **Type scale:** Use Tailwind defaults but lean toward larger headings (text-4xl to text-6xl for marketing heroes, text-2xl to text-3xl for section headers). Body at text-base (16px). Dashboard uses slightly tighter scale.

Button Styles

- **Primary:** Background `#A6F84C`, text `#0B0B0F` (dark on green). Rounded-lg (8px). Font-weight 600. Hover: `#BCFF6E`. This is the main CTA style used on fullstack.com ("Get Started", "Hire Now").
- **Secondary:** Background transparent, border `#2A2A30`, text white. Hover: background `#1E1E24`.
- **Ghost:** No background, no border, text `#9CA3AF`. Hover: text white.
- **Danger:** Background `#F87171`, text white.

Card Styles

- Background: `#16161C`
- Border: 1px solid `#2A2A30`
- Border-radius: 12px (rounded-xl)
- Hover: border color shifts to `#A6F84C` at 30% opacity, subtle translateY(-2px) lift
- Padding: 24px (p-6)

- Match the card rhythm on fullstack.com/talent/home — consistent spacing, clear hierarchy within each card

General Styling Notes

- All pages are dark mode. There is no light mode. The entire app is dark.
- Shadows are minimal — use border colors and subtle background shifts for depth instead of box-shadows (matching fullstack.com's flat-dark style).
- Icons: Use Lucide React icons. Stroke width 1.5. Match the icon style on fullstack.com (thin, clean line icons).
- Transitions: All hover/focus transitions at 150ms ease. Subtle, not bouncy.
- Focus rings: `#A6F84C` at 50% opacity with 2px offset. Ensures accessibility while matching brand.
- Scrollbars: Style dark (webkit-scrollbar with `#1E1E24` track and `#2A2A30` thumb).

Responsive

- Mobile-first for marketing pages (marketplace will be browsed on phones)
- Dashboard is desktop-first (sidebar collapses to sheet on mobile)
- Breakpoints: sm(640), md(768), lg(1024), xl(1280)
- Marketing pages should have the same expansive feel as fullstack.com — generous padding (px-6 to px-24 depending on breakpoint), max-w-7xl content containers, plenty of vertical rhythm between sections.

Execution Notes

- **Do not stub or mock anything that can use real Supabase data.** The seed data exists for a reason — wire everything to the database.
- **Server components by default.** Only add `"use client"` for interactive elements (forms, tabs, message input, filters).
- **Error handling:** Every Supabase query should handle errors gracefully. Show user-friendly error states, not blank screens.
- **Loading states:** Use `loading.tsx` files in route segments for Suspense boundaries. Keep them simple (skeleton cards or a spinner).
- **No placeholder "Lorem ipsum" anywhere.** All copy should be real, on-brand, and ready for a customer to read.

- **Commit frequently** with descriptive messages. One commit per major feature area.

Start with environment setup and database, then build marketing pages (home → outcomes catalog → outcome detail), then auth, then dashboard. This ordering means we have something demo-able at each checkpoint.